LOOMI CONNECT AI HACKATHON

Bloomreach x Bounteous x Kendra Scott

Developer Onboarding Kit

Personalization Performance Doctor

Version 1.0 | May 2026

Hackathon Build Window: 26 May - 2 Jun 2026

Demo Day: 4 Jun 2026

INTERNAL USE ONLY — DO NOT DISTRIBUTE

1. Hackathon Brief

1.1 What Is This Hackathon?

The Loomi Connect AI Hackathon is a virtual, asynchronous competition run by Bloomreach in June 2026. It challenges teams to build agentic AI workflows on top of Bloomreach's Loomi Connect platform — the MCP (Model Context Protocol) layer that exposes Bloomreach's intelligence as structured tools for AI agents.

Theme: Build agents that turn data, decisions, and actions into continuous customer outcomes.

The winning team wins a fully sponsored trip to Edge Summit (Los Angeles or London) for up to 5 members.

1.2 Key Dates

Milestone	Date
Idea Submission Deadline	May 22, 2026
Build Window Opens	May 26, 2026 (post kickoff)
Final Submission Deadline	Jun 2, 2026 — 4:00 PM PST
Demo Day	Jun 4, 2026

1.3 What Judges Evaluate

Five criteria, equally weighted at 20% each:

Criterion	What Judges Look For
Problem Relevance & Clarity	Real, meaningful problem. Clear user. Legible value proposition.
MCP Utilization & Depth	Loomi Connect MCPs used meaningfully — not just a background API call.
Agent Behaviour & Intelligence	Demonstrates understand → decide → act. Reasoning, not just retrieval.
Execution Quality & Feasibility	Sound architecture. Reliable demo. Credible path to production.
Innovation & Differentiation	Pushes beyond standard analytics or dashboard workflows.

1.4 Our Challenge Track

Primary Track: T2 — Engagement Intelligence & Workspace Diagnostics (with T3 overlap)

Track T2 tasks teams with building agents that help teams understand complex engagement setups. The primary MCP surface is Marketing MCP. Track T3 (Analytics Agents) overlaps because our solution also uses Analytics MCP to score A/B test coverage and generate fix recommendations.

1.5 What We Are Submitting

Six required artifacts:

- Project summary — 500 words, plain language, problem/solution/user/value
- Demo video — 5–6 minutes, end-to-end walkthrough
- Architecture overview — diagram showing UI, agent runtime, MCP surfaces, data flow
- MCP usage explanation — which surfaces, which tools, how deeply used, why
- Responsible design note — data handling, approval flow, what is simulated vs executed
- Team details — names, roles, Slack handles

2. Client Context — Kendra Scott

2.1 Who Is Kendra Scott?

Kendra Scott is a US-based fashion jewellery and lifestyle brand headquartered in Austin, TX. It is a premium DTC (direct-to-consumer) retailer operating across e-commerce, physical retail, and wholesale channels. The brand has a strong focus on personalization as a revenue lever — the CEO came from Gap Inc. (Athleta) and has explicitly set a vision for personalization at scale.

2.2 Their Technology Stack (Relevant to Us)

Platform	Role in Their Stack	Relevant to Our Build?
Bloomreach Discovery	Site search, product sorting, boost/bury rules, 1:1 personalization	YES — primary data source
Bloomreach Engagement	Email journeys, SMS, audience segments (on hold for full rollout)	YES — via Marketing MCP
Amperity	CDP — customer identity stitching, segment creation, pushes to Listrak	Awareness only for hackathon
Monetate	On-site A/B testing, product directs, banners, CRO	Reference context only
Snowflake	Data warehouse — all data lands here first	Background context
Listrak	Email/SMS blast platform (being evaluated for migration to Amperity)	Background context

2.3 Their Personalization Problem — In Their Own Words

Amanda Valdez — Digital Personalization Manager, Kendra Scott

"It's still very manual. We're telling [Bloomreach] what audience to go after. Right now we're working on customer journeys via email — I'm literally using a UTM parameter in the URL to set up audience groups."

"[The 1:1 personalization] results were not better. We ran a test for about a week and didn't see any positive traction. Most of our customers aren't logging in when they come to the site — they may log in once they get to cart."

This is the exact pain our solution must demonstrate solving. The diagnostic tool we build will surface why 1:1 personalization is underperforming — the root causes Amanda cannot currently see.

2.4 Key Stakeholders

Name	Role	Relevance to Build
Jared Miller	VP Digital / Personalization Lead, KS	Sponsor. Sets strategic direction. Wants "personalization at scale".
Amanda Valdez	Digital Personalization Manager, KS	Primary user persona for the demo. Day-to-day operator of Discovery.
Eric Buhman	Program Director, Bounteous	Hackathon team lead. Owns submission and coordination.
Jim Biermeier	VP Growth, Bounteous	Commerce architecture oversight. Bloomreach partnership lead.
SK (Saravana Kumar)	Senior Technical Director, Bounteous	Original technical ideation. Your technical reference for architecture decisions.

3. The Idea — Personalization Performance Doctor

3.1 The Problem We Are Solving

Kendra Scott has Bloomreach Discovery configured with 1:1 personalization enabled. It is underperforming. The merchandising team cannot identify why. The real root causes — low BRUID match rates, sparse AutoSegments, stale signals, rule conflicts, and poor A/B test coverage — are invisible from the front end.

Amanda and her team are flying blind. They run a test, it fails, they do not know what to fix first. This is not a tools problem — it is a diagnostics and decisioning problem.

3.2 Our Solution

The Personalization Performance Doctor is an AI diagnostic agent that:

- **Scores personalization health** across five dimensions into a single Personalization Readiness Score (PRS) from 0–100
- **Explains root causes** in plain English via a natural language interface — "why is my score low?"
- **Generates a ranked fix list** with estimated revenue impact per fix so Amanda knows what to act on first
- **Shows archetype-level results** — side-by-side search result comparison across four KS shopper personas
- **Keeps humans in control** — agent recommends, human approves before any action is taken

3.3 The Five PRS Scoring Dimensions

Dimension	What It Measures	Data Source	Weight
BRUID Match Rate	What % of shoppers are being recognized by Bloomreach across sessions	Discovery API	0–20 pts
AutoSegment Coverage	What % of shoppers fall into a meaningful behavioral segment	Marketing MCP	0–20 pts
Signal Freshness	How recent the behavioral signals feeding personalization are	Marketing MCP	0–20 pts
Rule Conflicts	How many active boost/bury rules are conflicting with each other	Discovery API	0–20 pts

Dimension	What It Measures	Data Source	Weight
A/B Test Coverage	What % of personalization decisions are being tested and measured	Analytics MCP	0–20 pts

Demo scenario target: PRS lands at 52/100 (Yellow). BRUID Match Rate is the flagged top fix. This matches the real KS problem Amanda described.

3.4 The Four KS Shopper Archetypes (Demo Personas)

Archetype	Behavioural Profile	Demo Query
Prospective	First visit. No login. No purchase history. Browsing broadly.	"necklace"
New Customer	Has purchased once. Recently logged in. Limited signal.	"yellow rose"
Gifting Shopper	High intent, seasonal. Searching for others. Price sensitive.	"necklace gift"
Returning VIP	Repeat purchaser. Known preferences. High LTV.	"necklace gift"

3.5 Why This Wins on the Judging Criteria

Criterion	How We Score Well
Problem Relevance	Direct quote from KS stakeholder. Real failing 1:1 personalization test. Specific user (Amanda).
MCP Utilization	Marketing MCP for segment/journey signals. Analytics MCP for A/B coverage. Discovery API for BRUID and rules.
Agent Behaviour	NL query handler triggers understand→decide→recommend flow. Agent selects which MCP tools to call based on query intent.
Execution Quality	PRS scoring engine is deterministic. Demo runs on synthetic JSON — reliable, no live data dependency.
Innovation	No one is surfacing a Personalization Readiness Score today. Fix list with revenue impact is genuinely novel.

4. High-Level Work Breakdown — Major Modules

The build is structured across five technical modules plus one supporting module. Each module is independent enough to be owned by a single developer, but they connect via defined interfaces. Understand all five before you touch code — your module depends on another's output schema.

Module	Name	What It Does	Owner Suggestion	Phase
M1	MCP Integration Layer	Authenticated connections to Marketing MCP, Analytics MCP, and Discovery API. Normalises all responses into a common JSON schema.	Dev 1	Build
M2	PRS Scoring Engine	Consumes M1's normalised JSON. Computes five sub-scores. Rolls up to 0–100 composite. Generates ranked fix list.	Dev 1 / Dev 2	Build
M3	Natural Language Interface	Accepts merchandiser plain-English queries. Triggers agent reasoning chain. Calls correct MCP tools. Returns LLM-generated explanation.	Dev 2	Build
M4	Front-End Dashboard	PRS scorecard UI (Module A), Shopper Archetype Simulator (Module B, static mockup), NL Chat Interface (Module C).	Dev 2 / Dev 3	Build
M5	Synthetic Data & Environment	Defines all test data: archetype profiles, PRS dimension scores, fix list content. Enables demo to run without live KS data.	Dev 1	Pre-build
M6	Submission Artifacts	Architecture diagram, demo script, project summary, MCP usage doc, responsible design note.	PD + Dev 1	Pre/Post build

4.1 Module Dependency Map

Build order matters. If M1 is not complete, M2 and M3 cannot be wired to live data. M5 exists precisely to unblock M2 and M3 early by providing synthetic schemas.

Dependency Chain

M5 (Synthetic Data) → unblocks → M2 and M3 development in parallel

M1 (MCP Integration) → feeds → M2 (PRS Engine) → feeds → M4 (Dashboard)
M1 (MCP Integration) → feeds → M3 (NL Interface) → feeds → M4 (Dashboard)
M4 (Dashboard) integrates M2 output + M3 output into unified UI
M6 (Artifacts) requires M1 architecture decisions + M4 demo to be stable

5. Detailed Work Breakdown — Task by Task

Every task below follows a four-field format: WHAT (definition), WHY (rationale), HOW (approach), CLAUDE (how AI can accelerate this specific task). Estimated hours are for a skilled mid-level developer.

PHASE 1: PRE-HACKATHON (Fri 22 May – Tue 26 May)

M5: Synthetic Data & Environment Setup

5.1 Define Synthetic Data Schema — KS Shopper Archetypes *(Est: 2 hrs)*

WHAT	Create the four shopper archetype behavioral profiles (prospective, new, gifting, returning) with associated behavioral signals: click history, session frequency, login status, purchase recency, segment membership.
WHY	The demo cannot depend on live KS data. Synthetic data that mirrors realistic KS shopper patterns lets you build and test all downstream modules (M2, M3, M4) before MCP access is confirmed. It also de-risks the demo — live data in a sandbox is unpredictable.
HOW	Define a JSON schema with fields: archetype_id, login_status, session_count, purchase_history, segment_list, last_signal_date, bruid_present. Populate four archetype objects. Store in /data/archetypes.json.
CLAUDE	Prompt Claude: "Given these four shopper archetypes for a premium jewellery brand [describe], generate a JSON file with realistic behavioral profiles. Include BRUID presence, segment membership, signal freshness dates, and click history for each." Claude will produce the full JSON. Review for KS realism.

5.2 Define Synthetic PRS Dimension Scores for Demo Scenario *(Est: 2 hrs)*

WHAT	Create the five dimension scores such that the composite PRS lands at 52/100 (Yellow) with BRUID Match Rate as the primary flagged issue (lowest sub-score).
WHY	The demo story requires a specific diagnostic outcome — "your 1:1 personalization is underperforming because your BRUID match rate is critically low." Every downstream module must render against this scenario. Defining it now means all developers are building toward the same demo state.
HOW	Create /data/prs_demo_state.json. Assign sub-scores: BRUID=8/20, AutoSegment=12/20, Signal Freshness=14/20, Rule Conflicts=10/20, A/B Coverage=8/20. Total=52/100. Add status labels (critical/warning/healthy) and fix priority ranks.
CLAUDE	Prompt Claude: "I need a JSON file representing a PRS diagnostic scenario for a jewellery retailer. The composite score should be 52/100. BRUID match rate and A/B coverage should be the lowest sub-scores. Generate the full JSON with sub-scores, status labels, and a ranked fix list with estimated revenue impact per fix." Review revenue estimates with CA before finalising.

5.3 Define Fix List Content — Top 3 Ranked Recommendations *(Est: 1.5 hrs)*

WHAT	Write the three fix recommendations that will appear in the dashboard fix list panel, with KS-specific descriptions and revenue impact estimates.
WHY	The fix list is the most visible output of the PRS engine. It needs to be specific to KS (not generic SaaS copy), credible enough that Amanda would actually act on it, and have revenue estimates that do not invite embarrassing scrutiny from judges.
HOW	For each fix: define dimension_linked, fix_title, plain_english_description, effort_level (Low/Medium/High), estimated_revenue_impact (expressed as % RPV lift or $ range). Example Fix 1: "Implement BRUID persistence across guest sessions — estimated 12–18% RPV lift on returning visitors."
CLAUDE	Prompt Claude: "For a mid-size DTC jewellery retailer running Bloomreach Discovery, write three specific personalization fix recommendations targeting BRUID match rate, A/B test coverage, and AutoSegment coverage. Each fix should include a plain-English description, effort level, and estimated revenue impact. Make the language actionable for a digital merchandising manager."

5.4 Set Up Dev Environment, Repo, and Branching Structure *(Est: 1.5 hrs)*

WHAT	Initialise the project repository with folder structure, environment variable templates, dependency files, and branching conventions.
WHY	Three developers working without a clear repo structure will create merge conflicts and integration failures during the build window. Fifteen minutes of setup saves hours of debugging.
HOW	Create repo with branches: main, dev, feature/m1-mcp, feature/m2-scoring, feature/m3-nl, feature/m4-ui. Root structure: /src (m1-mcp, m2-scoring, m3-nl, m4-ui), /data (synthetic JSON), /docs (architecture, demo script), /tests. Add .env.example with placeholders for all API keys.
CLAUDE	Prompt Claude: "Generate a complete project folder structure and package.json for a Node.js/React application that includes an MCP integration layer, a scoring engine, an NL query handler using the Anthropic API, and a React front-end dashboard. Include a README with setup instructions."

M1: MCP Surface Exploration & Feasibility

5.5 Obtain Bloomreach Sandbox Credentials and Confirm Access *(Est: 2 hrs)*

WHAT	Receive credentials from Bloomreach via the hackathon Slack channel. Authenticate against Marketing MCP and Analytics MCP. Confirm both endpoints are reachable and return tool lists.
WHY	This is the single biggest blocker in the entire project. If sandbox credentials arrive late or MCP endpoints are unreachable, the entire MCP integration layer is blocked. Escalate immediately in #loomi-connect if anything is wrong.

HOW	Check #sandbox-support and your team Slack channel for credentials. Use Postman or a simple Node script to authenticate. Run a starter prompt against each MCP. Document the tool list returned by each surface. If credentials are missing by Tue 26 May morning, tag @andrew-kumar and @peter-centgraf in #loomi-connect with "BLOCKED".
CLAUDE	Prompt Claude: "Write a Node.js script that authenticates with an MCP server using [auth method from credential email] and lists all available tools. Include error handling and log the full tool list to console." Paste actual error messages to Claude for debugging.

5.6 Explore Marketing MCP — Map Tools to PRS Dimensions *(Est: 3 hrs)*

WHAT	Authenticate, run starter prompts, document available tools, and map each tool output to its corresponding PRS scoring dimension (AutoSegment coverage, signal freshness).
WHY	You cannot build the MCP integration layer without knowing what the tools are named, what parameters they accept, and what shape their responses take. Discovery here directly informs M1 code.
HOW	For each tool in Marketing MCP: run it with a sample prompt, record the full JSON response, identify which fields map to AutoSegment coverage and signal freshness. Document in /docs/mcp-tool-map.md. Flag any gap where a needed field is not available — this triggers a synthetic data fallback.
CLAUDE	Prompt Claude: "Given this Marketing MCP tool response [paste response], identify which fields correspond to: audience segment membership count, segment coverage %, last signal update timestamp, and journey enrollment status. Map them to this PRS scoring dimension schema [paste schema]."

5.7 Explore Analytics MCP — Map Tools to PRS Dimensions *(Est: 3 hrs)*

WHAT	Authenticate, run starter prompts, document available tools, and map each tool output to the A/B test coverage PRS dimension and to revenue impact baseline data for the fix list.
WHY	A/B test coverage is one of the five PRS dimensions. If Analytics MCP cannot supply it, we need to fall back to synthetic data for that dimension. Knowing this early avoids a last-minute panic in the build phase.
HOW	Same approach as 5.6. Focus on: A/B test result retrieval, conversion rate by segment, engagement trend analysis. Document in /docs/mcp-tool-map.md under Analytics section.
CLAUDE	Same Claude pattern as 5.6 — paste tool responses, ask Claude to map fields to PRS dimensions.

5.8 Document MCP Limitations and Define Synthetic Fallbacks *(Est: 2 hrs)*

WHAT	For each PRS dimension where MCP cannot supply the required data in the sandbox, define the synthetic JSON substitute that will be used in the demo.

WHY	A hackathon sandbox is not a production environment. Some MCP tools may return empty data, mock data, or refuse to respond to certain queries. You must know this before the build window — not during it.
HOW	Create /docs/mcp-limitations.md. For each of the five PRS dimensions: mark as MCP-live or synthetic-fallback. For synthetic fallbacks, the data is already in /data/prs_demo_state.json (from task 5.2). The demo will run reliably regardless of MCP sandbox state.
CLAUDE	Prompt Claude: "Given this list of PRS scoring dimensions and the MCP tools we have confirmed access to, identify which dimensions cannot be sourced from MCP and generate the synthetic JSON fallback data for each. The fallback must match this schema: [paste schema]."

PHASE 2: HACKATHON BUILD (Wed 27 May – Tue 2 Jun)

M1: MCP Integration Layer

5.9 Implement Marketing MCP Authentication and Connection *(Est: 2 hrs)*

WHAT	Write the authenticated client for Marketing MCP. Confirm tool list is returned correctly. Log all tool names and confirm schemas match the exploration documentation.
WHY	This is the foundation of M1. Every subsequent MCP call depends on a working authenticated session. Do not proceed to building individual tool calls until this is solid.
HOW	Create /src/m1-mcp/marketing-client.js. Implement: authenticate(), listTools(), callTool(toolName, params). Use credentials from .env. Add retry logic (max 3 attempts, 1s backoff). Log tool list on successful connection.
CLAUDE	Prompt Claude: "Write a Node.js MCP client class for [Marketing MCP endpoint] that handles authentication, lists available tools, and exposes a callTool method. Use the Anthropic MCP SDK pattern. Include error handling, retry logic, and structured logging."

5.10 Implement Analytics MCP Authentication and Connection *(Est: 1.5 hrs)*

WHAT	Same as 5.9 but for Analytics MCP. Separate client file.
WHY	Marketing and Analytics MCP may have different auth flows or base URLs. Keep them as separate clients to isolate failures.
HOW	Create /src/m1-mcp/analytics-client.js. Mirror the pattern from 5.9.
CLAUDE	Prompt Claude: "Given this Marketing MCP client [paste marketing-client.js], generate an equivalent Analytics MCP client following the same pattern but for [Analytics MCP endpoint]."

5.11 Build MCP Calls for PRS Dimensions *(Est: 6 hrs)*

WHAT	Implement the specific tool calls for each of the five PRS dimensions: AutoSegment coverage (Marketing MCP), signal freshness (Marketing MCP), A/B coverage (Analytics MCP). BRUID and rule conflicts use Discovery API (tasks 5.12–5.13).
WHY	These five functions are the intelligence layer of the whole application. The PRS scoring engine (M2) consumes their outputs. Each function must return a normalised object matching the PRS schema — not the raw MCP response.
HOW	Create /src/m1-mcp/prs-data-fetcher.js with five exported async functions: fetchAutoSegmentCoverage(), fetchSignalFreshness(), fetchABCoverage(), one per dimension. Each function: calls the relevant MCP tool, parses the response, returns { dimension, raw_value, normalised_value, status, timestamp }.
CLAUDE	Prompt Claude: "Write a fetchAutoSegmentCoverage() function that calls the Marketing MCP tool [tool name from exploration doc], parses this response schema [paste schema], and returns an object with dimension, raw_value, normalised_value (0–20), and status (critical/warning/healthy). If the MCP call fails, fall back to this synthetic data: [paste from prs_demo_state.json]."

5.12 Build Discovery API Calls — BRUID Match Rate *(Est: 2 hrs)*

WHAT	Implement the Discovery API call that retrieves BRUID match rate data. This is supplementary to MCP — Discovery API is called directly, not through Loomi Connect MCP.
WHY	BRUID match rate is not available through Marketing or Analytics MCP. It requires a direct Discovery API call. This is explicitly called out in the judging submission — you must document which data comes from MCP vs direct API.
HOW	Create /src/m1-mcp/discovery-client.js. Implement fetchBRUIDMatchRate() using the Discovery REST API endpoint. Authenticate with Discovery API credentials (separate from MCP). Return normalised PRS dimension object.
CLAUDE	Prompt Claude: "Write a fetchBRUIDMatchRate() function using the Bloomreach Discovery REST API. Authenticate with [auth method], call [endpoint], parse the response to extract total_sessions, matched_sessions, and compute match_rate as a percentage. Return a normalised PRS object."

5.13 Build Discovery API Call — Active Rule Conflict Detection *(Est: 2 hrs)*

WHAT	Retrieve all active boost/bury rules from Discovery API and detect conflicts (two or more rules targeting the same query or product segment).
WHY	Rule conflicts are the fifth PRS dimension. They require fetching the active rule set and running conflict detection logic. This is not available via MCP.
HOW	Add fetchRuleConflicts() to /src/m1-mcp/discovery-client.js. Fetch active rules list. Conflict detection logic: flag any two rules that share a query AND a product. Count conflicts. Return normalised PRS dimension object.
CLAUDE	Prompt Claude: "Write a fetchRuleConflicts() function that fetches all active boost/bury rules from the Discovery API and detects conflicts. Two rules conflict if they target the same query term AND affect the same product. Return a count of conflicts and the normalised PRS sub-score (20 minus 2 points per conflict, floor 0)."

5.14 Build MCP Response Parser and Normaliser *(Est: 3 hrs)*

WHAT	A single normaliser function that takes any raw response from any of the five data-fetch functions and outputs a common PRS dimension JSON object.
WHY	M2 (PRS scoring engine) must consume a consistent schema regardless of whether the data came from Marketing MCP, Analytics MCP, or Discovery API. The normaliser is the contract between M1 and M2.
HOW	Create /src/m1-mcp/normaliser.js. Export normaliseDimension(raw, dimensionConfig). Input: raw API/MCP response + config specifying field mappings. Output: { dimension_id, raw_value, normalised_score (0–20), status, data_source, timestamp, is_synthetic }. Run unit tests against all five dimension types.
CLAUDE	Prompt Claude: "Write a normaliseDimension() function and a full set of Jest unit tests covering all five PRS dimensions. Each test should verify that a sample raw response from [dimension source] produces the correct normalised_score and status label."

M2: PRS Scoring Engine

5.15 Build Five PRS Dimension Scorers *(Est: 4 hrs)*

WHAT	One scorer function per dimension. Each takes a normalised dimension object from M1 and produces a sub-score (0–20) plus a status label.
WHY	The scorers translate raw normalised data into the weighted sub-scores that compose the final PRS. Each scorer needs its own thresholds (what raw value = critical, warning, healthy).
HOW	Create /src/m2-scoring/dimension-scorers.js. Export five functions: scoreBRUID(), scoreAutoSegment(), scoreSignalFreshness(), scoreRuleConflicts(), scoreABCoverage(). Each applies threshold logic and returns { dimension_id, score, max_score: 20, status, explanation }.
CLAUDE	Prompt Claude: "Write scoreBRUID() that takes a normalised BRUID dimension object and applies these thresholds: raw_value >= 0.7 = healthy (18–20pts), 0.4–0.69 = warning (10–17pts), < 0.4 = critical (0–9pts). Include a one-sentence explanation string for each band. Generate the full set of five scorer functions with equivalent threshold logic and Jest tests."

5.16 Build PRS Roll-Up — Composite Score and RAG Status *(Est: 2 hrs)*

WHAT	Combine all five sub-scores into a 0–100 composite score. Apply RAG (Red/Amber/Green) logic: < 50 = Red, 50–74 = Yellow/Amber, 75+ = Green.
WHY	The composite PRS is the single headline number that the dashboard shows. It must be deterministic, auditable, and produce exactly 52/100 on the demo scenario data.
HOW	Create /src/m2-scoring/prs-calculator.js. Export calculatePRS(dimensionResults). Input: array of five scorer outputs. Sum sub-scores. Apply RAG. Output: { composite_score, rag_status, dimensions: [...], generated_at }. Validate against demo scenario: must output 52 from the synthetic data.

CLAUDE	Prompt Claude: "Write calculatePRS() that sums five dimension sub-scores into a composite 0–100 score, applies RAG status logic, and returns a complete PRS result object. Include a Jest test that validates it outputs exactly 52/100 from this input: [paste prs_demo_state.json]."

5.17 Build Fix List Generator *(Est: 2 hrs)*

WHAT	Rank the top 3 fixes by estimated revenue impact based on which dimensions scored lowest. Each fix must include dimension, title, description, effort level, and revenue impact.
WHY	The fix list is what Amanda actually does with the PRS output. It converts a score into actions. A score of 52 with no fix list is an interesting number. A score of 52 with "Fix this first and recover 12% RPV" is a tool she will use every week.
HOW	Create /src/m2-scoring/fix-generator.js. Export generateFixList(prsResult). Logic: sort dimensions by score ascending, take bottom 3, map each to its pre-defined fix object from /data/fix-catalogue.json. Return ranked array. Fix catalogue is static data authored in task 5.3.
CLAUDE	Prompt Claude: "Write generateFixList() that takes a PRS result, ranks dimensions by score ascending, and maps the bottom three to fix objects from this catalogue: [paste fix catalogue JSON]. Return a ranked array with position, dimension, fix_title, description, effort, revenue_impact, action_label."

M3: Natural Language Interface Layer

5.18 Build NL Query Handler *(Est: 3 hrs)*

WHAT	Accept a plain-English query from a merchandiser ("why is my personalization score low?") and route it to the correct MCP tools and scoring functions.
WHY	This is the single most important component for the judging criterion "Agent Behaviour & Intelligence." A dashboard that displays a score is not an agent. An agent that understands a natural language question and decides which tools to call is an agent. This is what separates our submission from a glorified report.
HOW	Create /src/m3-nl/query-handler.js. Export handleQuery(queryText, currentPRSState). Steps: (1) classify intent (diagnosis, fix-request, dimension-drill, archetype-compare), (2) route to appropriate function, (3) gather context, (4) pass to LLM explainer. Use a simple intent classifier — keyword matching is acceptable for hackathon scope.
CLAUDE	Prompt Claude: "Write a query handler for a personalization diagnostic assistant. It should classify incoming queries into four intents: [list intents]. For each intent, define what context to gather from the PRS state. Include three representative test cases: 'why is my score low', 'what should I fix first', 'why is 1:1 personalization not working'."

5.19 Build Agent Reasoning Chain *(Est: 3 hrs)*

WHAT	Implement the understand → decide → recommend flow. The agent must inspect the PRS state, identify which MCP tools to call for additional context, assemble the evidence, and pass it to the LLM.

WHY	The judging rubric explicitly looks for reasoning, not just static output. The reasoning chain is the evidence of that. It must be visible in the UI — judges need to see which tools were called and why.
HOW	Create /src/m3-nl/reasoning-chain.js. Export runReasoningChain(intent, currentPRSState). Steps: (1) determine relevant dimensions from intent, (2) select MCP tools to call for live context, (3) execute tool calls, (4) assemble context object: { query, intent, prs_snapshot, mcp_evidence, fix_list }. Log each step — this log feeds the UI.
CLAUDE	Prompt Claude: "Write a reasoning chain for a personalization diagnostic agent. Given an intent and PRS state, it should: (1) identify relevant PRS dimensions, (2) select which MCP tools to call, (3) call them, (4) return an assembled evidence object. Show the chain of thought at each step. Include structured logging of each decision point for display in the UI."

5.20 Build LLM Prompt for PRS Explanation Generation *(Est: 3 hrs)*

WHAT	Design the system prompt and user prompt that takes the assembled evidence object and returns a plain-English explanation with reasoning, score breakdown, top fixes, and suggested next action.
WHY	The quality of the NL output is what Amanda and the judges actually see. A technically correct PRS score wrapped in generic AI language fails. The prompt must produce a concise, KS-specific, actionable explanation that sounds like a knowledgeable colleague.
HOW	Create /src/m3-nl/llm-explainer.js. Use Anthropic API (claude-sonnet-4-20250514). System prompt: establishes persona as a Bloomreach personalization expert familiar with the KS context. User prompt: injects PRS state + MCP evidence. Output structure: summary_sentence, score_breakdown (one line per dimension), top_3_fixes (title + one-sentence description each), suggested_next_action.
CLAUDE	Prompt Claude directly: "You are helping me write a system prompt for a personalization diagnostic AI assistant. The assistant explains Bloomreach PRS scores to a digital merchandising manager at a jewellery retailer. Write a system prompt that establishes expertise, specifies the output format (summary, score breakdown, fixes, next action), and avoids generic AI filler language. Then write the user prompt template that injects the PRS state and MCP evidence."

M4: Front-End Dashboard

5.21 Build Dashboard Shell — Layout and Navigation *(Est: 3 hrs)*

WHAT	Create the React application shell with three module tabs: Module A (PRS Scorecard), Module B (Archetype Simulator), Module C (NL Chat Interface).
WHY	All three modules share the same application shell. Getting the shell right early means Devs 2 and 3 can build their modules in parallel without layout conflicts.
HOW	Use React + Tailwind. Three-tab navigation. Responsive layout (1200px+ width target). Dark navy header with KS-appropriate colour palette (navy, teal, warm amber accents). Load synthetic data from /data/ on mount.

CLAUDE	Prompt Claude: "Generate a React application shell with Tailwind CSS for a B2B analytics dashboard. Three tabs: PRS Scorecard, Shopper Simulator, Ask the Doctor. Navy (#1B3A5C) header, teal (#0E7C7B) active tab indicator, clean white content area. Include a loading state. Use this data shape on mount: [paste prs_demo_state.json]."

5.22 Build Module A — PRS Scorecard *(Est: 5 hrs)*

WHAT	Display the composite PRS score (circular dial or gauge), RAG status indicator, five dimension sub-score breakdown, and fix list panel.
WHY	Module A is the primary diagnostic output. It must communicate the score, status, and top fixes at a glance. Judges will spend the most time looking at this.
HOW	Scorecard component: circular progress gauge (use recharts or a custom SVG). RAG colour coding: red < 50, amber 50–74, green 75+. Dimension panel: five rows, each showing dimension name, score bar, status icon. Fix list: three cards, each with rank badge, dimension tag, fix title, revenue impact, and "Review" button (triggers approval prompt).
CLAUDE	Prompt Claude: "Build a React PRS scorecard component. Include: (1) a circular gauge showing score 52/100 with amber colouring, (2) five dimension rows showing score bars and status icons using this data [paste dimensions from prs_demo_state.json], (3) three fix cards with rank badges and revenue impact using this data [paste fix list]. Use Tailwind only. No external charting libraries."

5.23 Build Module B — Shopper Archetype Simulator (Static Mockup) *(Est: 4 hrs)*

WHAT	Static side-by-side comparison showing personalised vs generic search results for each of the four KS archetypes. This is a mockup, not a live query.
WHY	The archetype simulator visualises what personalisation should look like when working correctly. It is illustrative. Judges understand mockups — they are explicitly permitted. Do not spend build time making this dynamic.
HOW	Build as a static React component. Four archetype selector tabs. Each tab shows two columns: "Personalised Results" and "Generic Results". Six product cards per column. Products are synthetic KS jewellery items (yellow rose earrings, gift necklaces, etc.) from /data/archetypes.json. Add a banner: "Illustrative — shows expected results when BRUID match rate reaches 70%+."
CLAUDE	Prompt Claude: "Generate a static React archetype simulator component. Four tabs: Prospective, New Customer, Gifting, Returning VIP. Each tab shows two columns of product cards. Use this synthetic product data: [paste archetype data]. Add a clearly labelled 'Illustrative' banner. Style with Tailwind to match the dashboard palette."

5.24 Build Module C — NL Chat Interface *(Est: 5 hrs)*

WHAT	A chat input box connected to the M3 NL layer. Displays the agent reasoning chain (tools called, MCP sources used) alongside the plain-English explanation output.
WHY	Module C is where the "agent behaviour" judging criterion is visually demonstrated. The reasoning chain display is not decorative — it shows judges that the agent is reasoning, not just pattern-matching.

HOW	Chat UI: input box at bottom, conversation history above, each agent response includes two sections: (1) collapsed "Reasoning trace" showing tools called and data sources, (2) expanded plain-English explanation. Seed the demo with one pre-loaded exchange: "why is 1:1 personalization not working?" → agent response.
CLAUDE	Prompt Claude: "Build a React chat interface component. Messages display in a scroll area. Each assistant message has an expandable 'Reasoning trace' panel showing which MCP tools were called. The main message shows the LLM explanation text. Style with Tailwind. Seed with this demo exchange: [paste query + response from demo scenario]."

5.25 Build Human Approval Prompt Component *(Est: 2 hrs)*

WHAT	A modal or inline prompt that appears when a user clicks "Review" on any fix recommendation. It presents the proposed action and requires Approve / Review Later / Dismiss before proceeding.
WHY	The judging rubric and starter kit both explicitly require a human approval step for business-impacting workflows. This is also the "Responsible Design" submission artifact. Build it visibly — it must appear in the demo video.
HOW	Create a reusable ApprovalModal React component. Props: action_title, action_description, estimated_impact, risk_level. Three buttons: Approve (teal), Review Later (grey), Dismiss (text link). On Approve: show a confirmation state "Action logged for review by your team." Log the approval to console (simulate write).
CLAUDE	Prompt Claude: "Write a React ApprovalModal component. It receives an action title, description, risk level, and estimated impact. Shows three buttons: Approve, Review Later, Dismiss. On Approve, transitions to a confirmation state. Include Tailwind styles matching this palette: [paste colours]. This component represents a human-in-the-loop approval gate for AI recommendations."

Integration, Testing, and Demo Hardening

5.26 End-to-End Integration Test *(Est: 4 hrs)*

WHAT	Wire all modules together: MCP layer → scoring engine → NL layer → dashboard UI. Run full flow on synthetic data. Confirm all components connect and produce expected output.
WHY	Modules built in isolation frequently fail when connected. Find integration failures during Day 3 of the build, not on Day 5.
HOW	Run the full flow from a single test script: (1) load synthetic data, (2) run all five dimension scorers, (3) calculate PRS, (4) generate fix list, (5) run a sample NL query through the reasoning chain and LLM explainer, (6) confirm dashboard renders correctly. Log each step.
CLAUDE	Prompt Claude: "Write an integration test script that runs the full PPD flow from synthetic data through scoring, fix generation, NL query handling, and LLM explanation. Assert: composite score = 52, top fix = BRUID, NL response is non-empty. Output a structured test report."

5.27 Demo Flow Dry Run and Critical Defect Fixes *(Est: 4 hrs)*

WHAT	Run the complete demo scenario end-to-end in a clean browser session, following the exact demo script. Identify and fix any issues that would break the demo during judging.
WHY	Demo failures during submission reviews are unrecoverable. The starter kit explicitly recommends backup screenshots and a recorded walkthrough as insurance. Do not skip this.
HOW	Follow demo script step by step. Record issues. Triage by impact on demo flow. Fix critical defects only — do not open new scope. Also record a backup screen capture walkthrough in case live demo fails during judging.
CLAUDE	Use Claude to diagnose any bugs that surface. Paste error messages and component code. Claude can debug React state issues, MCP parsing errors, and LLM prompt failures quickly.

6. How to Use Claude Effectively — Developer Guide

Claude is not a rubber duck. Used correctly, it is the equivalent of a senior developer who knows the full codebase. Used badly, it generates plausible-looking code that silently fails. These rules apply to this project.

6.1 The Right Mindset

- **Give Claude the schema, not the goal.** Do not say "build me an MCP client." Say "build me an MCP client that authenticates with this endpoint, returns tool lists in this format, and uses this error handling pattern. Here is the schema it must produce: [paste schema]."
- **Paste real inputs, not descriptions.** Claude generates better code from actual JSON response samples than from descriptions of what the JSON looks like. Always paste real or synthetic data.
- **Tell Claude what module you are in.** At the start of each session: "I am building M1 of the Personalization Performance Doctor. M1 is the MCP integration layer. Here is the current state of the normaliser.js file: [paste]. Now I need to..."
- **Review revenue estimates manually.** Claude will generate revenue impact numbers that sound reasonable. They need CA or senior team review before they appear in the demo. Do not let synthetic revenue estimates go unchecked.
- **Use Claude for debugging immediately.** Paste the full error message, the relevant file, and the failing test. Claude debugs faster than Stack Overflow.

6.2 High-Value Prompt Patterns

Task Type	Prompt Pattern
Generating a function from a schema	"Write [function name] that takes [input schema — paste it] and returns [output schema — paste it]. Apply this business logic: [rules]. Include Jest tests that validate [specific assertion]."
Debugging a failing test	"This test is failing: [paste test]. The function under test is: [paste function]. The error is: [paste error]. Identify the root cause and provide a fix."
Writing an LLM prompt	"Write a system prompt for an AI assistant that [role]. The assistant receives [input structure]. It must return [output structure]. Avoid [specific failure modes]. Optimise for [quality criterion]."
Generating synthetic data	"Generate a JSON file representing [entity]. It must contain [required fields]. The values should reflect [domain context]. Target this specific output state: [desired values]."
Building a React component	"Build a React component: [name]. Props: [list]. It displays [what]. Uses Tailwind only. Matches this colour palette: [hex values]. Handles these states: [loading, error, success]."

Task Type	Prompt Pattern
Architecture documentation	"Given this system: [paste architecture]. Write the MCP usage explanation section for the hackathon submission. It must answer: which surfaces, which tools, what value they provide, how deeply used."

6.3 What Claude Cannot Replace

- Sandbox exploration — Claude cannot authenticate with Bloomreach MCP on your behalf. You run the initial MCP calls. You paste the responses to Claude.
- KS stakeholder validation — Amanda or CA must sign off on fix list content and revenue estimates. Claude will generate plausible content. It does not know KS margins.
- Demo recording — Claude can write the demo script. You record it.
- Architecture sign-off — Claude can draft the architecture diagram spec. Senior team (SK or Jim) should review before the submission artifact is finalised.

7. Pre-Requisites — What Must Be Resolved Before You Build

Do not start Phase 2 build work until every P0 item below is confirmed. P1 items should be resolved by end of day on May 26.

7.1 P0 — Hard Blockers (Build Cannot Start Without These)

Item	What You Need	Where to Get It	Escalation Path
Bloomreach Sandbox Credentials	Login credentials for Bloomreach Engagement sandbox environment. Org ID, username, password.	Delivered to your team Slack channel by Bloomreach. Check #sandbox-support.	Tag @andrew-kumar + @peter-centgraf in #loomi-connect. Use #BLOCKED tag.
Marketing MCP Endpoint + Auth	Base URL, authentication method (API key / OAuth / Bearer), and any required headers for Marketing MCP.	Included in the sandbox credential email. Also available in Loomi Connect documentation.	Same escalation as above.
Analytics MCP Endpoint + Auth	Same as above for Analytics MCP.	Same source as above.	Same escalation.
Discovery API Credentials	Separate API credentials for Bloomreach Discovery (distinct from Engagement). Required for BRUID and rule conflict endpoints.	May be in the same credential package or may require separate request. Confirm with Eric.	Eric to raise with Jim / Bloomreach partnership contact.
Anthropic API Key	API key for claude-sonnet-4-20250514 for the NL explanation layer.	Team lead (Eric) to provide. Store in .env, never commit to repo.	If unavailable: AWS Bedrock Claude access is the fallback. Raise with Eric immediately.
Repo Access	All three developers added to the project repo with write access to their feature branches.	Team lead sets up repo and sends invites.	Slack Eric directly.

7.2 P1 — Required Before Phase 2 Build (Day 1 of Build Window)

Item	What You Need	Owner
MCP Tool Map Documentation	/docs/mcp-tool-map.md completed. Every available tool listed with response schema. Dimension-to-tool mapping confirmed.	Dev 1 (during Phase 1 exploration tasks 5.6–5.7)
Synthetic Data Files	/data/archetypes.json, /data/prs_demo_state.json, /data/fix-catalogue.json all committed to repo.	Dev 1 (tasks 5.1–5.3)
Dev Environment Running	Repo cloned, .env populated from .env.example, npm install passing, dev server starting without errors.	All developers (task 5.4)
LLM Model Confirmed	Decision made: Anthropic API vs AWS Bedrock vs other. .env.example updated with correct key names.	Dev 2 (task 5.4)
KS Demo Persona Confirmed	Amanda (or equivalent) confirmed as demo persona. Archetype names and query strings signed off.	Eric / PD to confirm with KS

7.3 P2 — Nice to Have (Do Not Block On These)

- AWS or GCP hosting credentials — only needed if you plan to host the demo externally. Running locally is acceptable.
- KS active contributor involvement — Amanda or Kayla contributing to the build. Useful for realism checks. Not required for the build itself.
- PayPal credentials — not relevant to our use case.

7.4 Slack Channels You Must Join

Channel	Purpose
#loomi-connect	All Bloomreach MCP technical questions. Primary support channel.
#sandbox-support	Credentials, access issues, environment problems. Post here if blocked.
#announcements	Official updates, deadline changes. Monitor daily.
#general	Cross-team discussion.
#team-[your-team-name]	Your team working channel. Daily check-ins posted here.
#submissions	Final submission questions and reminders.

Display name format in Slack: **First Name Last Name – Team Name**

8. Quick Reference

8.1 Key Terminology

Term	Definition
PRS	Personalization Readiness Score — the 0–100 composite score our tool generates.
BRUID	Bloomreach Unique ID — anonymous identifier Bloomreach uses to recognise a shopper across sessions. Low match rate = personalization cannot identify who is visiting.
AutoSegments	Audience groups Discovery builds automatically from shopper behaviour (e.g. "frequent buyers"). Low coverage = not enough shoppers are being grouped.
Loomi	Bloomreach's AI engine powering personalization and recommendations.
Loomi Connect	The MCP layer exposing Loomi intelligence as structured tools for AI agents. This is what our build consumes.
MCP	Model Context Protocol — a standard for exposing tool-based capabilities to AI agents.
Marketing MCP	Loomi Connect surface giving access to Engagement data: segments, journeys, campaign signals.
Analytics MCP	Loomi Connect surface giving access to performance data, A/B test results, trends.
Boost/Bury Rules	Manual overrides in Discovery that push specific products up or down in search results.
RAG Status	Red/Amber/Green — traffic light status applied to PRS composite score (Red <50, Amber 50–74, Green 75+).
Fix List	Ranked list of top 3 actions to improve PRS, generated by the scoring engine.
Archetype Simulator	Module B — static mockup showing personalised vs generic search results per shopper type.
Human Approval Step	Required UI component: before any AI-recommended action is taken, user must Approve/Review/Dismiss.

8.2 Critical File Locations

File / Directory	Purpose
/src/m1-mcp/	MCP integration layer — all MCP clients, Discovery API client, normaliser
/src/m2-scoring/	PRS scoring engine — dimension scorers, PRS calculator, fix list generator

File / Directory	Purpose
/src/m3-nl/	NL interface — query handler, reasoning chain, LLM explainer
/src/m4-ui/	React dashboard — all UI components
/data/archetypes.json	Synthetic shopper archetype profiles
/data/prs_demo_state.json	Synthetic PRS demo scenario (52/100, BRUID flagged)
/data/fix-catalogue.json	Fix recommendations with revenue estimates
/docs/mcp-tool-map.md	MCP exploration findings — tool names, schemas, dimension mappings
/docs/mcp-limitations.md	Which dimensions use live MCP vs synthetic fallback
/.env.example	Environment variable template — all API key placeholders

8.3 Submission Checklist

1. Project summary written (500 words, plain language, problem/solution/user/value)
2. Demo video recorded, uploaded, link tested in incognito — 5–6 minutes
3. Architecture diagram complete (UI, agent runtime, MCP surfaces, data flow, output, approval step)
4. MCP usage explanation written (which surfaces, which tools, depth of usage, value provided)
5. Responsible design note written (data handling, approval flow, simulated vs executed, limitations)
6. Team details submitted (names, roles, Slack handles, team lead identified)
7. All submission links verified accessible in incognito browser
8. Demo video uploaded minimum 24 hours before Jun 2 4pm PST deadline

Final Deadline

Jun 2, 2026 — 4:00 PM PST
Equivalent: approximately 5:30 AM IST on Jun 3, 2026
Upload demo video at least 24 hours before this time.
Test all links in incognito before submitting.

8.4 Who to Contact and When

Situation	Contact	Channel
MCP credentials not received	Andrew Kumar / Peter Centgraf	#loomi-connect (tag both, use #BLOCKED)

Situation	Contact	Channel
MCP endpoint returning errors	Andrew Kumar / Peter Centgraf	#loomi-connect
KS persona / demo scenario questions	Eric Buhman	Direct Slack message
Architecture review	SK (Saravana Kumar)	Direct Slack message
Revenue estimate validation	Jim Biermeier / CA	Direct Slack message
Submission portal questions	Hackathon team	#submissions
General technical questions	Claude (AI)	Claude.ai — paste context + question

---

Build something that makes reviewers say: this is a credible glimpse of where customer experience workflows are going.

— Bloomreach Hackathon Starter Kit